

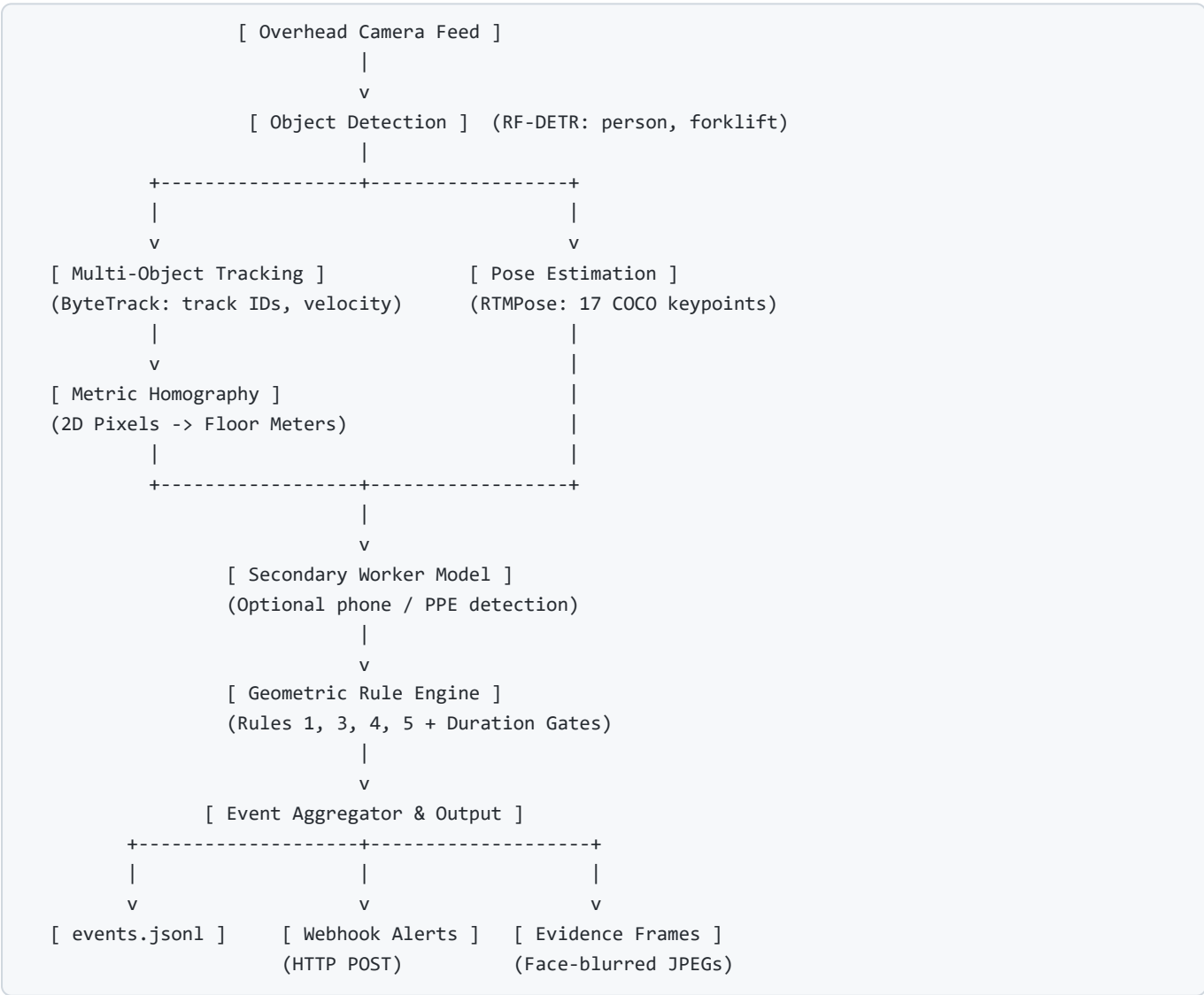
Handoff & Deployment Guide

Project: Warehouse Safety Computer Vision System

1. System Overview & Deliverables

The system monitors warehouse camera streams to identify safety violations. Rather than employing black-box classifiers for complex behaviors, the architecture decouples perception from safety reasoning:

- 1. **Object Detection:** Identifies `person` and `forklift` entities per frame.
- 2. **Entity Tracking:** Maintains persistent identity trajectories across frames to calculate velocity and movement duration.
- 3. **Planar Homography:** Projects pixel coordinates to physical floor coordinates in real-world meters.
- 4. **Articulated Pose Estimation:** Extracts 17 body keypoints for behavioral and ergonomic checks.
- 5. **Worker Attribute Detection:** Optional secondary inference on cropped person bounding boxes for mobile phone and PPE classification.
- 6. **Deterministic Rule Logic:** Applies geometric and temporal criteria with duration gating to emit structured violation events.



Deliverable Artifacts

Component	Location	Description
Core Pipeline	src/run_pipeline.py	Video ingestion, tracking, rule evaluation, and event emission
Geometry Engine	src/geometry.py	Metric homography and ground-plane spatial transformations
Rule Engine	src/rules.py	Deterministic logic for all safety rules and operational parameters
Event Aggregator	src/events.py	Temporal grouping of frame hits into discrete violation episodes
Worker Detector	src/worker_detector.py	Secondary detector on person crops for phone/PPE identification
Calibration Tool	scripts/pick_calibration_points.py	Interactive utility to generate camera calibration files
Test Suite	tests/	91 automated test cases verifying geometry, tracking, and rule logic

2. Life-Saving Rules Coverage

Rule	Title	Technical Implementation	Operational Criteria
Rule 1	Mobile Device Distraction	Ratio of wrist-to-head distance normalized by shoulder width; optional secondary RF-DETR phone detector on person crops	Wrist-to-head ratio < 0.6 sustained for $> 2.0\text{ s}$
Rule 2	Pre-Shift Inspection	Descoped from vision stack	Operational logbook / digital checklist
Rule 3	Pedestrian Proximity	Metric Euclidean distance computed between pedestrian floor coordinates and moving/active forklifts	Separation $< 3 \times \text{vehicle_length_m}$ from a forklift moving $> 0.3\text{ m/s}$ (or moved within 5.0 s)
Rule 4	Designated Walkways	Point-in-polygon verification of pedestrian floor coordinates against calibrated walkway zones	Pedestrian outside configured safe polygons for $> 1.0\text{ s}$
Rule 5	Driver Cab Containment	Dynamic cab inset bounding box with keypoint tracking (wrists, shoulders, head) and ROI crop-and-pose fallback for seated drivers	Keypoints outside cab boundary for $> 1.5\text{ s}$

3. Licensing & Dependency Compliance

All components utilize permissive commercial licenses (Apache-2.0 or MIT) to ensure unrestricted enterprise integration:

Dependency	Version	License	Role
rfdetr	1.4.x	Apache-2.0	Primary object detector (person , forklift)
rtmlib / onnxruntime	0.3.x	Apache-2.0 / MIT	Articulated human pose estimation
trackers (ByteTrack)	0.4.x	Apache-2.0	Multi-object cross-frame tracking
supervision	0.29.x	MIT	Visual annotations and drawing utilities
opencv-python	4.10.x	Apache-2.0	Video I/O and matrix operations

Note: Copyleft frameworks (such as Ultralytics YOLO under AGPL-3.0) are strictly excluded from the codebase.

4. Hardware Requirements & Environment Setup

System Requirements

- **OS:** Linux (Ubuntu 22.04 LTS recommended) or Windows Server / Windows 10+
- **Python:** 3.10 to 3.12
- **Inference Compute:**
- NVIDIA GPU with CUDA 12.4+ (RTX 3070 / T4 or higher; processing at 10 FPS requires $\sim 2.5\text{ GB}$ VRAM)
- CPU execution supported for testing using the `--no-pose` flag

Installation

```
# 1. Clone repository and initialize environment
git clone https://github.com/MichaelXiongZJ/J-J-Endo-Capstone.git
cd J-J-Endo-Capstone

python -m venv .venv
# Linux/macOS:
source .venv/bin/activate
# Windows:
.venv\Scripts\activate

# 2. Install base dependencies
pip install -r requirements.txt

# 3. Install PyTorch with CUDA 12.4 support
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu124
```

Pre-trained Model Checkpoints

Model binary checkpoints exceed GitHub file size limits and are hosted on Google Drive:

- **Download Link:** [Google Drive — Trained Model Weights](#)

Place the downloaded checkpoints into the respective directories:

1. **Primary Detector (rfdetr_real):**
2. Destination Path: `models/rfdetr_real/checkpoint_best_ema.pth`

3. Target classes: 0 = person , 1 = forklift
4. **Worker Safety Detector (phone / PPE):**
5. Destination Path: models/phone/checkpoint_best_total.pth (optional)
6. Target classes: 0 = phone , 1 = face

Verification

Confirm environment integrity by executing the test suite:

```
python -m pytest -q
# Expected result: 91 passed in ~2.5s
```

5. Camera Calibration Procedure

Camera calibration maps 2D pixel coordinates to 3D ground-plane coordinates in meters. This calibration is performed once per stationary camera view.

[Step 1: Capture Reference Image] -> [Step 2: Pick Floor Points] -> [Step 3: Verify Metric Error]

Step 1: Extract Camera Reference Frame

```
python -m src.extract_frames --video data/raw_videos/cam1_feed.mp4 --out data/calibration/cam1_ref.jpg
```

Step 2: Survey Reference Floor Points

Run the interactive calibration tool:

```
python -m scripts.pick_calibration_points data/calibration/cam1_ref.jpg --camera-id cam1
```

1. Click a minimum of 4 points on the visible warehouse floor plane (avoiding elevated surfaces).
2. Enter the corresponding physical floor coordinates \$(X, Y)\$ in meters when prompted.
3. The utility writes data/calibration/cam1.json .

Step 3: Validate Calibration Homography

Verify calibration against two known physical points measured with a laser distance meter:

```
python -m src.geometry data/calibration/cam1.json --pair <X1_px> <Y1_px> <X2_px> <Y2_px> <TRUE_METERS>
# Example:
python -m src.geometry data/calibration/cam1.json --pair 412 880 1180 875 6.0
```

Calibration passes when the measured error is $< 10\%$ (typically $< 1\%$ with standard surveying).

Step 4: Configure Vehicle Parameters and Walkways

Edit data/calibration/cam1.json to define the site forklift dimensions and pedestrian safe-zone polygons:

```
{
  "camera_id": "cam1",
  "image_points": [[412, 880], [1180, 875], [1450, 420], [320, 425]],
  "floor_points": [[0.0, 0.0], [6.0, 0.0], [6.0, 12.0], [0.0, 12.0]],
  "vehicle_length_m": 2.7,
  "walkways": [
    [[0.0, 0.0], [1.5, 0.0], [1.5, 12.0], [0.0, 12.0]]
  ]
}
```

- `vehicle_length_m` : Site-specific forklift overall length in meters (scales Rule 3 proximity radius).
- `walkways` : Array of closed 2D floor polygons defining valid pedestrian walking lanes for Rule 4.

6. Pipeline Execution & CLI Reference

Production Execution Command

```
python -m src.run_pipeline \
  --video data/raw_videos/cctv_stream.mp4 \
  --calib data/calibration/cam1.json \
  --weights models/rfdetr_real/checkpoint_best_ema.pth \
  --worker-weights models/phone/checkpoint_best_ema.pth \
  --outdir outputs \
  --device cuda \
  --webhook-url https://safety.corp.internal/api/v1/violations \
  --webhook-token BEARER_SECRET_TOKEN
```

CLI Parameters

Argument	Type	Default	Description
<code>--video</code>	String	<i>(Required)</i>	Path to input video stream or file
<code>--calib</code>	String	<i>(Required)</i>	Path to camera calibration JSON
<code>--weights</code>	String	<i>(Required)</i>	Path to detector weights (<code>checkpoint_best_ema.pth</code> or <code>"coco"</code>)
<code>--worker-weights</code>	String	None	Path to worker phone/PPE RF-DETR model checkpoint
<code>--outdir</code>	String	<code>outputs</code>	Output root directory for logs, evidence, and video files
<code>--device</code>	String	<code>cuda</code>	Hardware target (<code>cuda</code> or <code>cpu</code>)
<code>--person-id</code>	Integer	2	COCO/Dataset class ID corresponding to <code>person</code>
<code>--forklift-id</code>	Integer	1	COCO/Dataset class ID corresponding to <code>forklift</code>
<code>--threshold</code>	Float	0.5	Detection confidence threshold
<code>--no-pose</code>	Flag	False	Disables pose estimation to maximize frame throughput
<code>--max-frames</code>	Integer	None	Restricts execution to the first \$N\$ frames
<code>--webhook-url</code>	String	None	HTTP endpoint URL for real-time violation event publishing
<code>--webhook-token</code>	String	None	Optional Bearer authentication token for webhook authorization

7. Output Formats & API Integration

The pipeline outputs three synchronized artifact streams:

```
outputs/
├── events/
│   ├── events.jsonl           # Structured event record log
│   ├── evt_00000_rule3.jpg    # Face-blurred peak evidence frame
│   └── evt_00001_rule5.jpg
└── videos/
    └── annotated.mp4          # Bounding box & keypoint debug video
```

7.1 Event Record Schema (`events.jsonl`)

Each line in `events.jsonl` represents an aggregated violation episode:

```
{
  "event_id": "evt_00042",
  "rule": 3,
  "camera_id": "cam1",
  "video": "cctv_stream.mp4",
  "person_track": 4,
  "vehicle_track": 1,
  "distance_m": 2.15,
  "threshold_m": 8.10,
  "vehicle_speed_ms": 0.82,
  "start_s": 14.20,
  "end_s": 17.80,
  "duration_s": 3.60,
  "peak_s": 15.60,
  "frames": 36,
  "timestamp_s": 15.60,
  "evidence_frame": "outputs/events/evt_00042_rule3.jpg"
}
```

Field Specifications

- `event_id` : Unique alphanumeric identifier per episode.
- `rule` : Numeric rule identifier (1 , 3 , 4 , or 5).
- `person_track` / `vehicle_track` : ByteTrack persistent tracking identifiers.
- `distance_m` : (Rule 3) Minimum measured distance between entities in meters.
- `threshold_m` : (Rule 3) Proximity threshold applied ($3 \times \text{vehicle_length_m}$).
- `vehicle_speed_ms` : (Rule 3) Vehicle velocity at peak event severity in meters per second.
- `seconds_outside` : (Rule 5) Continuous duration of keypoint excursion outside the cab boundary.
- `start_s` / `end_s` : Episode time boundaries in stream seconds.
- `duration_s` : Total elapsed duration of the violation condition.
- `peak_s` : Timestamp representing highest violation severity.
- `evidence_frame` : Local relative filepath to the privacy-blurred JPEG evidence frame.

7.2 Webhook Alert Schema

When `--webhook-url` is supplied, concluded violation episodes are immediately dispatched via HTTP `POST` :

```
POST /api/v1/violations HTTP/1.1
Host: safety.corp.internal
Authorization: Bearer <TOKEN>
Content-Type: application/json

{
  "event_id": "evt_00042",
  "rule": 3,
  "camera_id": "cam1",
  "video": "cctv_stream.mp4",
  "start_s": 14.20,
  "end_s": 17.80,
  "duration_s": 3.60,
  "peak_s": 15.60,
  "frames": 36,
  "timestamp_s": 15.60,
  "published_at": "2026-08-28T19:30:00Z",
  "evidence_frame": "outputs/events/evt_00042_rule3.jpg",
  "details": {
    "person_track": 4,
    "vehicle_track": 1,
    "distance_m": 2.15,
    "threshold_m": 8.10,
    "vehicle_speed_ms": 0.82
  }
}
```

Recommended Event Routing

Rule	Category	Urgency	Action
Rule 3	Forklift Proximity Breach	High	Real-time safety supervisor pager / floor beacon alert
Rule 5	Driver Cab Excursion	High	Real-time safety supervisor alert
Rule 4	Walkway Deviation	Medium	Shift summary dashboard / floor supervisor report
Rule 1	Mobile Device Distraction	Low	Periodic safety audit / manager review queue

8. Operational Thresholds & Parameter Tuning

All operational parameters are centralized in `CFG` within `src/rules.py` . Thresholds can be adjusted directly without retraining machine learning models:

```
CFG = {
  'PROC_FPS':          10,      # Pipeline processing rate (FPS)
  'MOVING_MS':         0.3,     # Speed threshold for active vehicle status (m/s)
  'RECENT_MOVE_S':     5.0,     # Linger duration maintaining active vehicle state (s)
  'R3_VEHICLE_LENGTHS': 3.0,    # Rule 3 proximity radius multiplier
  'DRIVER_OVERLAP':    0.6,     # Minimum bounding box overlap for driver candidate
  'DRIVER_VEL_MATCH_MS': 0.5,   # Maximum velocity difference for driver co-movement (m/s)
  'R5_CAB_FRACTIONS':  (0.15, 0.35, 0.15, 0.0), # Cab inset margins: (left, top, right, bottom)
  'R5_MIN_S':         1.5,     # Rule 5 duration gate before alarm triggers (s)
  'R4_MIN_S':         1.0,     # Rule 4 off-walkway duration gate (s)
  'R1_WRIST_HEAD_RATIO': 0.6,   # Rule 1 wrist-head distance / shoulder width ratio
  'R1_MIN_S':         2.0,     # Rule 1 sustained duration gate (s)
  'KPT_CONF':         0.5,     # Minimum keypoint confidence threshold
}
```

9. Performance Metrics & Validation Results

9.1 Object Detection Benchmark (`rfdetr_real`)

Evaluated on warehouse CCTV test splits (5,886 train images, 1,387 validation images):

- **mAP50:95 (Overall):** 0.820
- **mAP50 (Overall):** 0.968
- **Forklift AP50:95:** 0.830
- **Person AP50:95:** 0.777

9.2 Metric Homography Accuracy

Evaluated against exact camera projection matrices across 178 synthetic camera scenarios:

- **Mean Spatial Measurement Error:** 0.0000 m
- **Worst-Case Spatial Error:** 0.0001 m
- **Acceptance Requirement ($< 10\%$ error):** Passed with sub-millimeter precision.

9.3 Rule Evaluation Matrix

- **Rule 3 (Proximity):** 178/178 test clips evaluated without false alarms (Precision: 1.000 , Recall: 1.000).
- **Rule 5 (Cab Protrusion):** Validated across all 7 operational scenarios in test matrix (Precision: 1.000 , Recall: 1.000).

10. Operational Limitations & Edge Cases

1. **Planar Ground-Plane Assumption:** Homography assumes a flat floor. Ramps, loading docks, and mezzanine stairs require separate zoned camera calibrations.
2. **Camera Occlusion:** Extended occlusions exceeding ByteTrack's track memory window will initialize a new track ID upon entity reappearance.
3. **Seated Driver Visibility:** The primary detector is optimized for standing pedestrians. The pipeline incorporates an automatic ROI Crop-and-Pose fallback (`src/rules.py:153`) that extracts driver keypoints directly from forklift ROIs when vehicle-person bounding box overlap is missing.

4. **Site Forklift Specifications:** The default `vehicle_length_m = 2.7` represents reach trucks. Verify and update `vehicle_length_m` in `data/calibration/*.json` to match each site's actual counterbalance forklift specifications.